

Experiment 'p_4_1_c_powers_of_two_array' Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 3

Hard fails (resolution): 1

Soft fails (resolution): 0

Verification attempts: 2

Problem Specification

Problem name: p_4_1_c_powers_of_two_array

Natural language statement: null

Method signature: p_4_1_c_powers_of_two_array() returns (powers: array<int>)

Ensures

- powers.Length == 21
- forall i :: 0 <= i < powers.Length ==> powers[i] == pow(2, i)

Functional Code Given

```
function pow(base: int, exp: int): int
  requires 0 <= exp
  decreases exp
{
  if exp == 0 then 1 else base * pow(base, exp - 1)
}
```

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_4_1_c_powers_of_two_array()` returns `(powers: array<int>)`

The method should respect the following contract:

`ensures powers.Length == 21, ensures forall i :: 0 <= i < powers.Length ==> powers[i] ==`
`↪ pow(2, i)`

The contract uses the following dafny code:

```
function pow(base: int, exp: int): int
  requires 0 <= exp
  decreases exp
{
  if exp == 0 then 1 else base * pow(base, exp - 1)
}
```

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
`↪` surround it. Do not show the signature nor contract. You must not use 'assume' nor
`↪` 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnLCO7aNoJgYRaHkz4wOAuO0M9a93

Model: gpt-5.5-2026-04-23

Created at: 1780651932

Finish reason: stop

Usage: 434 tokens in, and 3889 tokens out

```
{
  powers := new int[21](i =>
    if i == 0 then 1
    else if i == 1 then 2
    else if i == 2 then 4
    else if i == 3 then 8
    else if i == 4 then 16
    else if i == 5 then 32
    else if i == 6 then 64
```

```

    else if i == 7 then 128
    else if i == 8 then 256
    else if i == 9 then 512
    else if i == 10 then 1024
    else if i == 11 then 2048
    else if i == 12 then 4096
    else if i == 13 then 8192
    else if i == 14 then 16384
    else if i == 15 then 32768
    else if i == 16 then 65536
    else if i == 17 then 131072
    else if i == 18 then 262144
    else if i == 19 then 524288
    else 1048576);
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```

problem_data/p_4_1_c_powers_of_two_array/program.dfy(11,0): Error: a postcondition could
    ↪ not be proved on this return path

```

```

|
11 | {
| ~

```

```

problem_data/p_4_1_c_powers_of_two_array/program.dfy(10,9): Related location: this is the
    ↪ postcondition that could not be proved

```

```

|
10 | ensures forall i :: 0 <= i < powers.Length ==> powers[i] == pow(2, i)
| ~~~~~~

```

Dafny program verifier finished with 2 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the
 ↪ Dafny body, including the curly braces that surround it. Do not show the signature
 ↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnLEHZaBZmqJ3c4RiYhcPdbAn2KhE

Model: gpt-5.5-2026-04-23

Created at: 1780652049

Finish reason: stop

Usage: 903 tokens in, and 1317 tokens out

```

{
    powers := new int[21];
    powers[0] := 1;

```

```

assert powers[0] == pow(2, 0);

var i := 1;
while i < powers.Length
  invariant powers.Length == 21
  invariant 1 <= i <= powers.Length
  invariant forall j :: 0 <= j < i ==> powers[j] == pow(2, j)
{
  powers[i] := 2 * powers[i - 1];
  assert 0 <= i - 1;
  assert powers[i - 1] == pow(2, i - 1);
  assert pow(2, i) == 2 * pow(2, i - 1);
  assert powers[i] == pow(2, i);
  assert forall j :: 0 <= j < i + 1 ==> powers[j] == pow(2, j) by {
    if j < i {
      assert powers[j] == pow(2, j);
    } else {
      assert j == i;
      assert powers[j] == pow(2, j);
    }
  }
  i := i + 1;
}
}

```

Program → GenAI

When using dafny resolve, the below error is emitted and resolve fails:

```

problem_data/p_4_1_c_powers_of_two_array/program.dfy(28,9): Error: unresolved identifier:
  ↪ j
  |
28 | if j < i {
  | ^

problem_data/p_4_1_c_powers_of_two_array/program.dfy(29,22): Error: unresolved identifier
  ↪ : j
  |
29 | assert powers[j] == pow(2, j);
  | ^

problem_data/p_4_1_c_powers_of_two_array/program.dfy(29,35): Error: unresolved identifier
  ↪ : j
  |
29 | assert powers[j] == pow(2, j);
  | ^

problem_data/p_4_1_c_powers_of_two_array/program.dfy(31,15): Error: unresolved identifier
  ↪ : j
  |
31 | assert j == i;

```

```

| ~
problem_data/p_4_1_c_powers_of_two_array/program.dfy(32,22): Error: unresolved identifier
  ↪ : j
|
32 | assert powers[j] == pow(2, j);
| ~

problem_data/p_4_1_c_powers_of_two_array/program.dfy(32,35): Error: unresolved identifier
  ↪ : j
|
32 | assert powers[j] == pow(2, j);
| ~

6 resolution/type errors detected in program.dfy

Correct the error in code by altering only the method body. Produce and show only the
  ↪ Dafny body, including the curly braces that surround it. Do not show the signature
  ↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

```

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnLEtntZ0nR4PhKdHgy2eoVuhsHxQ

Model: gpt-5.5-2026-04-23

Created at: 1780652087

Finish reason: stop

Usage: 1714 tokens in, and 1359 tokens out

```

{
  powers := new int[21];
  powers[0] := 1;
  assert powers[0] == pow(2, 0);

  var i := 1;
  while i < powers.Length
    invariant powers.Length == 21
    invariant 1 <= i <= powers.Length
    invariant forall j :: 0 <= j < i ==> powers[j] == pow(2, j)
  {
    powers[i] := 2 * powers[i - 1];
    assert 0 <= i - 1;
    assert powers[i - 1] == pow(2, i - 1);
    assert i != 0;
    assert pow(2, i) == 2 * pow(2, i - 1);
    assert powers[i] == pow(2, i);

    assert forall j :: 0 <= j < i + 1 ==> powers[j] == pow(2, j) by {
      forall j | 0 <= j < i + 1
        ensures powers[j] == pow(2, j)
      {

```

```

        if j < i {
            assert powers[j] == pow(2, j);
        } else {
            assert j == i;
            assert powers[j] == pow(2, j);
        }
    }
}
i := i + 1;
}
}

```

Final Program

```

function pow(base: int, exp: int): int
    requires 0 <= exp
    decreases exp
{
    if exp == 0 then 1 else base * pow(base, exp - 1)
}

method p_4_1_c_powers_of_two_array() returns (powers: array<int>)
    ensures powers.Length == 21
    ensures forall i :: 0 <= i < powers.Length ==> powers[i] == pow(2, i)
{
    powers := new int[21];
    powers[0] := 1;
    assert powers[0] == pow(2, 0);

    var i := 1;
    while i < powers.Length
        invariant powers.Length == 21
        invariant 1 <= i <= powers.Length
        invariant forall j :: 0 <= j < i ==> powers[j] == pow(2, j)
    {
        powers[i] := 2 * powers[i - 1];
        assert 0 <= i - 1;
        assert powers[i - 1] == pow(2, i - 1);
        assert i != 0;
        assert pow(2, i) == 2 * pow(2, i - 1);
        assert powers[i] == pow(2, i);

        assert forall j :: 0 <= j < i + 1 ==> powers[j] == pow(2, j) by {
            forall j | 0 <= j < i + 1
                ensures powers[j] == pow(2, j)
            {
                if j < i {
                    assert powers[j] == pow(2, j);
                } else {
                    assert j == i;

```

```
        assert powers[j] == pow(2, j);
    }
}
}
i := i + 1;
}
}
```

Total Token Usage

Input tokens: 3051

Output tokens: 6565

Reasoning tokens: 5633

Sum of ‘total tokens’: 9616

Experiment Timings

Overall Experiment started at 1780651931850, ended at 1780652126501, lasting 194651ms (194.65 seconds)

Iteration #1 started at 1780651931853, ended at 1780652047894, lasting 116041ms (116.04 seconds)

Iteration #2 started at 1780652047894, ended at 1780652086616, lasting 38722ms (38.72 seconds)

Iteration #3 started at 1780652086617, ended at 1780652126501, lasting 39884ms (39.88 seconds)